

Boas Práticas de Desenvolvimento de Software

Ronierison Maciel

FICR

2026



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: ronierison.maciел@p.ficr.edu.br

GitHub: <https://github.com/0x03c1>

- 1 Módulo 1 – Qualidade de Código: Legibilidade e Manutenibilidade
- 2 Módulo 2 – Princípio DRY e Funções: Evitando Repetição
- 3 Módulo 3 – Controle de Versão com Git: Fundamentos e Boas Práticas
- 4 Módulo 4 – Tratamento de Erros: Robustez e Segurança em C
- 5 Módulo 5 – Testes de Software: Verificando Corretude em C
- 6 Módulo 6 – Documentação: Comunicando Decisões em C
- 7 Módulo 7 – Revisão Geral e Projeto Final Integrador

Ao final desta aula, você será capaz de:

- Compreender o que é qualidade de código e por que ela importa.
- Aplicar boas práticas de nomenclatura e organização em C.
- Refatorar trechos de código com baixa qualidade em versões mais legíveis.

Pense antes de responder

- Imagine que você precisa **retomar um código em C** que escreveu há **6 meses**. O que você espera encontrar?
- Por que programadores costumam dizer que “**código é lido 10x mais do que escrito**”?
- Já tentou entender o código de outra pessoa em C e simplesmente **desistiu**? O que tornava tão difícil?

Compartilhe suas respostas em duplas por 2 minutos. Depois, abrimos para a turma.

O que é Qualidade de Código?

Código de baixa qualidade

- Nomes sem significado: x, tmp2, aux.
- Funções enormes que fazem tudo.
- Comentários que apenas repetem o código.
- Lógica duplicada em vários lugares.
- Sem verificação de erros de entrada.

Código de alta qualidade

- Nomes expressivos: calcular_media, idade_usuario.
- Funções pequenas com **uma única responsabilidade**.
- Comentários explicam o **porquê**, não o quê.
- Lógica centralizada e reutilizável.
- Entradas validadas e erros tratados.

Regra de ouro

“Qualquer tolo pode escrever código que um computador entende. Bons programadores escrevem código que humanos entendem.” — Martin Fowler

Ruim

```
1 float f(float *v, int n) {  
2     float s = 0;  
3     for (int i = 0; i < n; i++)  
4         s += v[i];  
5     return s / n;  
6 }
```

Bom

```
1 float calcular_media(  
2     float *notas,  
3     int quantidade) {  
4     float soma = 0.0f;  
5     for (int i = 0; i < quantidade; i++)  
6         soma += notas[i];  
7     return soma / quantidade;  
8 }
```

O que mudou?

- Nome da função descreve **o que ela faz**.
- Parâmetros e variáveis têm nomes que **revelam intenção**.
- Código muito mais fácil de **testar e depurar**.

Convenções da linguagem C

- **Funções e variáveis:** use snake_case — calcular_imc, nome_usuario.
- **Constantes e macros:** use MAIUSCULAS_COM_UNDERSCORE — #define MAX_ALUNOS 40.
- **Tipos (typedef struct):** use PascalCase — typedef struct {...} ContaCorrente;
- **Booleanos simulados:** prefixe com eh_ ou is_ — eh_valido, is_ativo.
- **Evite abreviações obscuras:** nome_completo em vez de nmCmpl.

Anti-padrão comum em 1.º período

Nomear variáveis com a, b, aux, temp fora de contextos matemáticos é um dos erros mais comuns e mais prejudiciais à legibilidade em C.

Comentário ruim (óbvio)

- `/* incrementa i */` antes de `i++`
- `/* retorna 0 */` antes de `return 0;`
- Comentários que apenas repetem o código em português.

Comentário bom (explica o porquê)

- Explica **decisões de design** não óbvias.
- Documenta **regras de negócio** complexas.
- Alerta sobre **comportamentos inesperados** ou limitações.

Exemplo de bom comentário em C

```
/* Multiplica por 0.9 pois o cliente tem desconto VIP de 10% */  
valor_final = preco * 0.9f;
```

Desafio (15 min em duplas)

Vocês receberão um trecho de código C com problemas de qualidade. Sua tarefa é **refatorá-lo** aplicando as boas práticas vistas.

Entregável:

- 1 Liste **todos os problemas** encontrados no código original.
- 2 Reescreva o código com **nomes adequados**, **funções menores** e **comentários úteis**.
- 3 Explique **cada mudança** que fez e por quê.

Critério de qualidade

O código refatorado deve ser lido por alguém que **nunca viu o original** e entender o que ele faz sem precisar de explicações externas.

Passos

- 1 Cada dupla troca seu código refatorado com outra dupla.
- 2 A dupla revisora aponta: **o que ficou bom?** e **o que ainda pode melhorar?**
- 3 Simulem um **code review** como se estivessem em uma empresa de software.
- 4 Devolvam o código com comentários de revisão escritos no próprio arquivo .c.

Reflexão pós-atividade:

- Você conseguiu entender o código da outra dupla sem pedir explicações?
- O que foi mais fácil de revisar? O que foi mais difícil?

Síntese da aula

- **Código limpo** não é luxo: é responsabilidade profissional. Código ruim custa tempo e dinheiro.
- Bons nomes, funções pequenas e comentários relevantes são as ferramentas mais simples e mais impactantes.
- **Escrever código é comunicação**: você está se comunicando com outros desenvolvedores e com o “você do futuro”.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Aplicar o princípio DRY (*Don't Repeat Yourself*) em C.
- Criar funções bem estruturadas com responsabilidade única.
- Organizar código em arquivos `.h` e `.c` separados.

Situação real

- Imagine que você escreveu a **mesma lógica de validação em 8 lugares** do sistema. Uma regra muda — você precisa alterar nos 8 lugares. Esqueceu um. **O sistema produz resultados errados silenciosamente.**
- Já copiou e colou um bloco de código em C em vez de criar uma função? O que aconteceu quando precisou mudar?
- Por que iniciantes tendem a **repetir código em vez de abstrair**?

Don't Repeat Yourself

Cada **pedaço de lógica ou conhecimento** deve ter uma única representação no sistema. Duplicação é a raiz de muitos problemas: bugs multiplicados, manutenção cara e inconsistências silenciosas.

Números do mercado

Estudos da indústria apontam que **até 30%** do tempo de desenvolvimento é gasto em retrabalho causado por **duplicação de código e falta de abstração**.

Consequências da violação do DRY

- Corrigir um bug em um lugar e esquecer nos outros.
- Código difícil de testar (muito para cobrir).
- Mudanças de regra viram pesadelo de manutenção.

Violando DRY

```
1 /* Repetido em 3 lugares */
2 float d1 = preco1 * 0.1f;
3 float pf1 = preco1 - d1;
4
5 float d2 = preco2 * 0.1f;
6 float pf2 = preco2 - d2;
7
8 float d3 = preco3 * 0.1f;
9 float pf3 = preco3 - d3;
```

Aplicando DRY

```
1 float aplicar_desconto(
2     float preco,
3     float percentual) {
4     return preco - (preco * percentual);
5 }
6
7 /* Uso: uma só linha por preco */
8 float pf1 = aplicar_desconto(
9     preco1, 0.1f);
10 float pf2 = aplicar_desconto(
11     preco2, 0.1f);
```

Vantagem imediata

Se o desconto mudar de 10% para 15%, você altera **em um único lugar**.

Single Responsibility Principle aplicado a funções em C

Uma função deve fazer **uma única coisa** e fazê-la bem. Se você precisa usar “e” para descrever o que ela faz, ela provavelmente deve ser dividida.

Função sobrecarregada

- `processar_ler_exibir_salvar()`
- Impossível testar cada parte isoladamente.
- Um bug afeta tudo ao mesmo tempo.
- Difícil de reutilizar partes.

Funções focadas

- `ler_dados_aluno()`
- `calcular_media_aluno()`
- `exibir_resultado()`
- `salvar_em_arquivo()`

calculadora.h — interface

Declara apenas as **assinaturas** das funções. É o “contrato público” do módulo.

```
#ifndef CALCULADORA_H
#define CALCULADORA_H

float somar(float a, float b);
int dividir(float a, float b,
            float *res);

#endif
```

calculadora.c — implementação

Contém **apenas** a lógica das funções declaradas no .h.

```
#include "calculadora.h"

float somar(float a, float b) {
    return a + b;
}
```

Por que isso importa?

Separar interface de implementação permite **trocar a implementação** sem alterar quem usa o módulo — base da modularidade profissional em C.

Desafio (10 min em duplas)

Analise o código C fornecido e identifique todas as violações do princípio DRY.

Entregável:

- 1 Liste cada **trecho duplicado** encontrado.
- 2 Proponha como **abstrair** cada duplicação em uma função.
- 3 Reescreva o código **sem repetição**, com protótipo declarado no `.h`.

Passos

- 1 Você receberá uma função em C com mais de 60 linhas que faz múltiplas coisas.
- 2 Identifique as **responsabilidades distintas** dentro dela.
- 3 Quebre em **funções menores** com nomes claros, declaradas no `.h`.
- 4 A função original deve se tornar apenas uma **orquestradora** das menores.

Reflexão pós-atividade:

- O código ficou mais fácil de entender? Por quê?
- Quantas funções você criou? Isso é muito ou pouco para o problema?

Síntese da aula

- **DRY** não é sobre não copiar e colar: é sobre não ter dois lugares no código que representem **a mesma verdade**.
- Funções pequenas e focadas são mais fáceis de **testar, depurar e reutilizar** — especialmente em C.
- Separar `.h` e `.c` é o primeiro passo para **modularidade profissional** em sistemas C.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Entender o que é controle de versão e por que é indispensável.
- Usar os comandos básicos do Git no fluxo de trabalho diário.
- Escrever mensagens de commit claras e organizadas.

Você já passou por isso?

- Arquivo chamado `trabalho_final_v3_agora_vai.c`?
- Deletou uma função que funcionava para “**melhorar**” e depois precisou recuperar?
- Duas pessoas editando o mesmo `.c` e uma **sobrescrevendo o trabalho da outra**?

Controle de versão resolve exatamente esses problemas. Git é o padrão da indústria há mais de 15 anos.

Conceito

Git é um **sistema de controle de versão distribuído** que registra cada mudança feita no código ao longo do tempo. Permite **voltar no tempo**, trabalhar em paralelo e colaborar sem conflitos.

Números

O GitHub hospeda mais de **420 milhões de repositórios** e mais de **100 milhões de desenvolvedores** usam Git como ferramenta principal de trabalho.

Benefícios no dia a dia

- Histórico completo de **quem mudou o quê e quando**.
- Trabalhar em funcionalidades novas **sem afetar o código estável**.
- Colaboração simultânea de **múltiplos desenvolvedores**.
- Integração com GitHub, GitLab e Bitbucket.

Os 4 comandos que você usará todos os dias

- 1 **git init** ou **git clone**: inicia ou clona um repositório.
- 2 **git add**: prepara arquivos para o commit (*staging area*).
- 3 **git commit**: salva um snapshot das mudanças com uma mensagem.
- 4 **git push** / **git pull**: sincroniza com o repositório remoto.

```
1 # Fluxo completo de uma mudanca em projeto C
2 git add calculadora.c calculadora.h
3 git commit -m "feat: adiciona divisao com verificacao de zero"
4 git push origin main
```

Commits ruins

- `git commit -m "alterações"`
- `git commit -m "fix"`
- `git commit -m "asdfghjkl"`
- `git commit -m "tentando de novo"`

Commits bons (Conventional Commits)

- `feat:` adiciona funcao `calcular_imc`
- `fix:` corrige divisao por zero em `media.c`
- `refactor:` extrai validacao para funcao separada
- `docs:` adiciona Doxygen em `pilha.h`

Prefixos Conventional Commits

`feat` (nova funcionalidade) | `fix` (correção de bug) | `refactor` | `docs` | `test` | `chore` (manutenção)

O conceito de ramo de desenvolvimento

Uma **branch** é uma linha independente de desenvolvimento. Crie uma branch para uma nova funcionalidade, desenvolva com segurança e depois **una ao código principal** quando estiver pronta.

Branches comuns em equipes

- **main**: código em produção/entrega.
- **develop**: integração de novas funcionalidades.
- **feature/nome-da-feature**: desenvolvimento isolado.
- **hotfix/nome-do-bug**: correção urgente.

Regra de ouro

Nunca faça commits diretamente na branch **main** em um projeto colaborativo. Sempre use branches e **Pull Requests** para revisão.

Desafio (20 min em duplas)

Criem e configurem seu primeiro repositório colaborativo no GitHub com código em C.

Passo a passo:

- 1 Crie um repositório público no GitHub com **README.md**.
- 2 Clone o repositório com `git clone`.
- 3 Crie uma branch: `feature/calculadora-basica`.
- 4 Implemente `calculadora.h` e `calculadora.c` com as 4 operações básicas.
- 5 Faça um commit semântico por operação implementada.
- 6 Faça push e abra um **Pull Request** para a branch `main`.
- 7 Revise o PR do colega e **aprove** ou **solicite alterações**.

Síntese da aula

- Git não é opcional: é a **habilidade técnica mais cobrada** em processos seletivos de tecnologia.
- Boas mensagens de commit são **documentação automática** da evolução do projeto.
- Branches permitem **experimentação segura**: errar em uma branch não quebra o código de ninguém.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Compreender as formas de sinalizar e tratar erros em C.
- Usar códigos de retorno, `errno` e `assert` adequadamente.
- Criar funções robustas que validam entradas e comunicam falhas claramente.

Cenas do cotidiano em C

- Você já viu um programa em C travar com **Segmentation Fault** sem nenhuma mensagem útil?
- O que acontece quando `malloc()` falha e você **não verifica o retorno**? O programa continua e corrompe memória silenciosamente.
- O que é pior: um programa que **aborta com mensagem clara** ou um que **continua com dados inválidos na memória**?

Erros de Compilação

- Detectados pelo gcc.
- O executável não é gerado.
- *Ex: tipo incompatível, ; faltando.*

Erros de Runtime

- Ocorrem durante a execução.
- *Ex: divisão por zero, ponteiro nulo, buffer overflow.*
- Frequentemente causam **Segmentation Fault**.

Erros de Lógica

- Os mais perigosos: o programa **executa sem travar** mas produz resultado errado.
- *Ex: índice off-by-one, overflow silencioso de inteiro.*

Sem tratamento

```
1 float dividir(float a, float b) {  
2     /* UB se b == 0 */  
3     return a / b;  
4 }
```

Com código de retorno

```
1 #define ERRO_DIVISAO_ZERO -1  
2 #define SUCESSO 0  
3  
4 int dividir(float a, float b,  
5             float *resultado) {  
6     if (b == 0.0f)  
7         return ERRO_DIVISAO_ZERO;  
8     *resultado = a / b;  
9     return SUCESSO;  
10 }
```

Convenção POSIX

Funções que retornam **0 para sucesso** e **valor negativo para erro** são padrão em sistemas Unix/Linux — aprenda a seguir essa convenção desde o início.

Regra absoluta: sempre verifique retornos de funções críticas

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     /* Verificando malloc */
6     int *vetor = malloc(100 * sizeof(int));
7     if (vetor == NULL) {
8         fprintf(stderr,
9             "Erro: memoria insuficiente.\n");
10        return EXIT_FAILURE;
11    }
12
13    /* Verificando fopen */
14    FILE *arq = fopen("dados.txt", "r");
15    if (arq == NULL) {
16        fprintf(stderr,
17            "Erro: nao foi possivel abrir o arquivo.\n");
18        free(vetor);
19        return EXIT_FAILURE;
20    }
21
22    /* uso seguro ... */
23    free(vetor);
24    fclose(arq);
25    return EXIT_SUCCESS;
26 }
```

O que é assert?

`assert(expr)` aborta o programa com mensagem clara se `expr` for falsa. Ideal para **invariantes internas** que *nunca* deveriam ser violadas em código correto.

```
1 #include <assert.h>
2
3 float calcular_media(
4     float *notas, int n) {
5     assert(notas != NULL);
6     assert(n > 0);
7     float soma = 0.0f;
8     for (int i = 0; i < n; i++)
9         soma += notas[i];
10    return soma / n;
11 }
```

assert vs. validação de entrada

assert: para **bugs internos** — nunca deve disparar em produção.

if + return/exit: para **entradas externas** do usuário ou de arquivos — podem falhar em uso normal e devem ser tratadas graciosamente.

Desafio (20 min em grupos de 3)

Vocês receberão um sistema de cadastro em C que **não trata nenhum erro**. Sua tarefa é torná-lo robusto.

Entregável:

- 1 Identifique **todos os pontos de falha**: ponteiros não verificados, divisões, `scanf` sem limite de buffer.
- 2 Adicione **verificação de retorno** em todos os `malloc`, `fopen` e `scanf`.
- 3 Defina e use **constantes de erro** com `#define` ou `enum`.
- 4 Teste com: número negativo onde se espera positivo, arquivo inexistente, string muito longa.

Critério de qualidade

O sistema não pode **nunca gerar Segmentation Fault ou comportamento indefinido**, independente da entrada.

Síntese da aula

- Em C, **não há exceções automáticas**: você é responsável por verificar cada retorno crítico.
- A regra de ouro: **nunca confie em uma entrada sem validá-la** — seja do usuário, de um arquivo ou da memória.
- Tratamento de erros em C é também uma questão de **segurança**: buffer overflows e use-after-free são portas de entrada para ataques reais.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Entender os tipos de testes e onde cada um se aplica.
- Escrever testes unitários em C com assert e estruturas simples de teste.
- Relacionar testes com qualidade, segurança e confiança no código.

Pense nas consequências

- Em 1996, o foguete **Ariane 5** explodiu 37 segundos após o lançamento por um bug de conversão de tipo não testado — prejuízo de **US\$ 370 milhões**.
- Em 2014, o bug **Heartbleed** em OpenSSL (código C) expôs dados de milhões de servidores por **2 anos** sem ser detectado.
- Qual seria o impacto de um bug não testado no **firmware do seu futuro empregador**?

Os três níveis

- **Testes unitários (base):** testam funções isoladas. Rápidos, baratos, muitos.
- **Testes de integração (meio):** testam interação entre módulos .c, arquivos e dispositivos.
- **Testes de sistema (topo):** validam o comportamento completo. Mais lentos e caros.

Proporção recomendada

70% unitários
20% integração
10% sistema

Quanto mais alto na pirâmide, mais caro para escrever e mais lento para executar.

```
1 /* calculadora.c */
2 float somar(float a, float b) { return a + b; }
3
4 int dividir_seguro(float a, float b, float *res) {
5     if (b == 0.0f) return -1;
6     *res = a / b;
7     return 0;
8 }
```

```
1 /* test_calculadora.c */
2 #include <assert.h>
3 #include <stdio.h>
4 #include "calculadora.h"
5
6 void test_somar_positivos(void) {
7     assert(somar(2.0f, 3.0f) == 5.0f);
8     printf("[OK] test_somar_positivos\n");
9 }
10
11 void test_dividir_por_zero(void) {
12     float res;
13     assert(dividir_seguro(10.0f, 0.0f, &res) == -1);
14     printf("[OK] test_dividir_por_zero\n");
15 }
```

```
1 /* test_calculadora.c (main de testes) */
2 int main(void) {
3     printf("=== Rodando testes ===\n");
4     test_somar_positivos();
5     test_dividir_por_zero();
6     printf("=== Todos os testes passaram! ===\n");
7     return 0;
8 }
```

Compilando e rodando

```
gcc -o test_calc test_calculadora.c calculadora.c && ./test_calc
```

Frameworks alternativos para C

Para projetos maiores, considere: **Unity** (padrão em sistemas embarcados), **CUnit** ou **Criterion** — todos seguem o mesmo padrão de casos de teste isolados.

Para qualquer função em C, sempre teste

- **Caso feliz (happy path):** entrada válida, resultado esperado.
- **Casos de borda (edge cases):** zero, array vazio, valor máximo/mínimo do tipo, ponteiro NULL.
- **Entradas inválidas:** valor negativo onde se espera positivo, string sem terminador `\0`.
- **Comportamentos de erro:** a função retorna o código de erro correto?

A pergunta que guia os testes em C

“O que acontece com esta função se o ponteiro for NULL? E se n for 0? E se for negativo?” Responder sistematicamente é o coração do teste em C.

Desafio (20 min em duplas)

Vocês receberão uma calculadora básica em C. Sua missão é **cobri-la com testes**.

Entregável:

- 1 Escreva um arquivo `test_calculadora.c` com funções de teste para: somar, subtrair, multiplicar, dividir_seguro.
- 2 Cubra pelo menos **3 casos por operação**: happy path, borda e erro.
- 3 Compile e execute: todos os testes devem imprimir [OK].
- 4 **Bônus**: Encontre um bug já existente no código usando seus testes.

Critério de sucesso

Nenhum teste deve causar Segmentation Fault (isso significaria que o teste em si está mal escrito).

Síntese da aula

- **Testar é parte do desenvolvimento**, não uma etapa opcional — especialmente em C, onde erros de memória são silenciosos e catastróficos.
- Testes unitários simples com assert já oferecem **grande retorno** com baixo custo de implementação.
- O maior benefício dos testes: **coragem para refatorar** — se os testes passam, o comportamento foi preservado.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Distinguir os tipos de documentação e para quem cada um se destina.
- Escrever cabeçalhos de funções no padrão Doxygen.
- Criar um README e Makefile profissionais para projetos em C.

Dilema clássico do desenvolvedor C

- Você já tentou usar uma biblioteca C sem documentação? Apenas os arquivos `.h`? O que aconteceu?
- Já precisou entender um projeto legado sem nenhum comentário nos `.h`? Quanto tempo perdeu?
- “O código é a documentação” — isso é verdade ou desculpa para não documentar?

O arquivo `.h` bem comentado é a API pública do seu módulo. Quem usa seu código só precisa ler o `.h`.

Documentação Interna

- **Cabeçalhos Doxygen no .h:** descrevem parâmetros, retornos e exemplos.
- **Comentários no .c:** explicam decisões de implementação não óbvias.
- **Constantes documentadas:** `#define` com comentário explicando o valor.

Documentação Externa

- **README.md:** apresentação, compilação e uso.
- **Doxygen HTML:** gerado automaticamente dos comentários.
- **Makefile:** documenta como compilar e testar.
- **Changelog:** histórico de versões e mudanças.

```
1 /**
2  * @brief Calcula o Índice de Massa Corporal (IMC).
3  *
4  * @param peso      Peso em quilogramas. Deve ser positivo.
5  * @param altura    Altura em metros. Deve ser positivo.
6  * @param imc       Ponteiro para armazenar o resultado.
7  *                  Não pode ser NULL.
8  *
9  * @return 0 em caso de sucesso.
10 * @return -1 se peso ou altura forem <= 0.
11 * @return -2 se o ponteiro imc for NULL.
12 *
13 * @example
14 *     float resultado;
15 *     calcular_imc(70.0f, 1.75f, &resultado);
16 *     // resultado ~= 22.86
17 */
18 int calcular_imc(float peso,
19                 float altura,
20                 float *imc);
```

```
1 CC      = gcc
2 CFLAGS  = -Wall -Wextra -std=c11 -g
3
4 all: programa
5
6 programa: main.o calculadora.o
7     $(CC) $(CFLAGS) -o programa main.o calculadora.o
8
9 calculadora.o: calculadora.c calculadora.h
10    $(CC) $(CFLAGS) -c calculadora.c
11
12 main.o: main.c calculadora.h
13    $(CC) $(CFLAGS) -c main.c
14
15 test: test_calculadora
16     ./test_calculadora
17
18 test_calculadora: test_calculadora.o calculadora.o
19    $(CC) $(CFLAGS) -o test_calculadora \
20        test_calculadora.o calculadora.o
21
22 clean:
23     rm -f *.o programa test_calculadora
```

Um bom README para projeto C deve conter

- 1 **Nome e descrição** do projeto em uma frase clara.
- 2 **Dependências:** compilador, versão do padrão C (C11), bibliotecas externas.
- 3 **Como compilar:** comando `make` ou `gcc` exato.
- 4 **Como executar:** exemplos com argumentos e saída esperada.
- 5 **Como testar:** como rodar os testes e interpretar a saída.
- 6 **Estrutura de arquivos:** o que cada `.h` e `.c` faz.

Impacto no mercado

Recrutadores e tech leads avaliam o GitHub dos candidatos. Um repositório com **README bem escrito e Makefile funcional** destaca o profissional imediatamente.

Desafio (20 min em grupos de 3)

Vocês receberão um projeto C funcional mas **sem nenhuma documentação**. Sua missão é documentá-lo completamente.

Entregável:

- 1 Adicione **cabeçalhos Doxygen** em todas as funções do arquivo `.h`.
- 2 Adicione comentários internos no `.c` explicando decisões de implementação.
- 3 Escreva um **README.md** com os 6 elementos obrigatórios.
- 4 Escreva um **Makefile** com alvos `all`, `test` e `clean`.
- 5 **Bônus:** Execute `doxygen` e abra o HTML gerado no navegador.

Síntese da aula

- O arquivo .h bem documentado com Doxygen é a **API pública do módulo**: quem usa não precisa ler a implementação.
- README e Makefile bem escritos são a diferença entre um projeto **usado e abandonado**.
- O equilíbrio: documente **decisões e contratos de interface**; deixe o código expressar a **implementação**.

Dúvidas? Reflexões finais?

Ao final desta aula, você será capaz de:

- Integrar todas as boas práticas estudadas em um único projeto em C.
- Avaliar criticamente código de terceiros à luz dos conceitos aprendidos.
- Apresentar um produto de software com qualidade profissional.

O que cobrimos nesta disciplina

- **Módulo 1:** Qualidade de código em C — nomes, funções, comentários.
- **Módulo 2:** DRY, SRP e modularização com `.h/.c`.
- **Módulo 3:** Git — controle de versão e colaboração com commits semânticos.
- **Módulo 4:** Tratamento de erros — códigos de retorno, assert, validação de ponteiros.
- **Módulo 5:** Testes unitários em C com assert e frameworks como Unity.
- **Módulo 6:** Documentação Doxygen, README e Makefile.

*Esses conceitos se **reforçam mutuamente**: código limpo é mais fácil de testar; código testado é mais seguro de refatorar; código documentado é mais fácil de manter.*

Desafio (individual ou dupla)

Desenvolva um **sistema de gerenciamento de alunos** em C — cadastro, consulta, cálculo de médias e persistência em arquivo — aplicando **todas as boas práticas** da disciplina.

Requisitos obrigatórios:

- 1 Código seguindo padrões de **nomenclatura e Clean Code em C**.
- 2 **Modularização**: mínimo de 3 pares `.h/.c` separados.
- 3 **Repositório GitHub** com commits semânticos e histórico claro.
- 4 **Tratamento de erros** em todos os `malloc`, `fopen` e entradas do usuário.
- 5 Arquivo `test_sistema.c` com **mínimo de 10 casos de teste**.
- 6 **README.md** completo, **Makefile** funcional e **Doxygen** no `.h`.

Pontuação

- Qualidade e legibilidade do código C: **2,0 pts**
- Modularização .h/.c e DRY: **1,5 pts**
- Git com commits semânticos: **1,5 pts**
- Tratamento de erros e robustez: **1,5 pts**
- Testes automatizados (mín. 10 casos): **2,0 pts**
- Documentação, README e Makefile: **1,5 pts**

Critério eliminatório

Projetos que causem **Segmentation Fault** ou **comportamento indefinido** nas entradas de teste do professor terão a nota de robustez zerada automaticamente.

Formato da apresentação (10 min por equipe)

- 1 **Demo ao vivo:** compilar com `make` e executar o sistema (3 min).
- 2 **Code walkthrough:** mostrar um módulo `.h/.c` que demonstra boas práticas (3 min).
- 3 **Mostrar o GitHub:** histórico de commits, README, execução dos testes (2 min).
- 4 **Perguntas da turma e do professor** (2 min).

Dica de apresentação

Prepare-se para explicar **por que separou os módulos dessa forma**. *“Separei a leitura de arquivo da lógica de negócio para que os testes unitários não precisem de arquivo de verdade”* é uma ótima resposta.

O que o mercado espera de um desenvolvedor júnior em C?

- Que escreva código que **outros possam entender e manter** — sem precisar depurar com `printf` cegamente.
- Que saiba usar **Git como ferramenta de colaboração**, não só de backup.
- Que nunca entregue código com **ponteiros não verificados ou memory leaks** óbvios.
- Que escreva **testes mínimos** para as funções críticas que desenvolve.
- Que **documente a interface pública** para que colegas usem o módulo sem ler a implementação.

Síntese final

- Boas práticas não são **regras burocráticas**: são **acordos de civilidade** entre desenvolvedores.
- Em C, onde o programador controla cada byte, a **disciplina de boas práticas é ainda mais crítica**: os erros são silenciosos e as consequências são sérias.
- O profissional mais valorizado não é o que escreve o código mais rápido — é o que escreve o código que **funciona, pode ser mantido e não assusta os colegas**.

“O código que você escreve revela como você pensa. Pense com clareza.”

Obrigado pela turma! Dúvidas finais?